

Spring Boot — Lecture 2

First Hello World App + IP, Port, DNS & Spring Initializr

1. Goal of This Lecture

Build a minimal Spring Boot app that returns **"Hello World"** in the browser. We won't understand the syntax yet — the intent is to feel how fast Spring Boot sets everything up so you can jump straight to business logic.

What we hit	Where	Expected response
GET /hello	localhost:8080/hello	"Hello World" string displayed in browser

2. IP Address

Every device on a network has a unique **IP address**. Any client anywhere in the world that wants to reach your computer must send data to this specific IP address.

To find your own IP: Google 'what is my ip' → whatismyip.com → copy your IPv4 address.

3. Port Number

One IP address can run many applications simultaneously (Chrome, WhatsApp, Spotify, Spring Boot all at once). The OS uses the **port number** to route incoming network data to the correct application.

Application	Port	Notes
Spring Boot App	8080	Default — set by Spring Boot / Tomcat
HTTP traffic	80	Standard — browser auto-appends this
HTTPS traffic	443	Standard secure — browser auto-appends this
Chrome / WhatsApp / etc.	varies	Each app gets its own unique port

Standard ports (80 / 443) are auto-appended by the browser — you never type them. For local development you must explicitly type :8080 in the URL.

4. Full Client → Server Request Flow (Real World)

For a real website like coderarmyarmy.in the request goes through several steps:

Step	What Happens
1 — DNS Lookup	Browser asks DNS server: 'What is the IP of coderarmyarmy.in?' → gets the server IP address.
2 — Port attach	Browser auto-appends port 443 (HTTPS) or 80 (HTTP) to the resolved IP address.
3 — Packet travels	Data packet travels over the internet to the server's IP address on that port.
4 — Reverse Proxy	Maps the public-facing port (443) to the internal app port (8080) where Spring Boot runs.
5 — Spring Boot	Receives the request on port 8080, processes it, and returns the HTTP response.

Reverse Proxy (e.g. Nginx):

Sits between client and your Spring Boot app. Maps external port (443) to internal app port (8080). Both the reverse proxy and the Spring Boot app together are called 'the server'.

For localhost (our case in this lecture):

Term	Meaning
127.0.0.1	Fixed loopback IP — means 'this machine itself'

Term	Meaning
localhost	Alias for 127.0.0.1 — easier to type
localhost:8080/hello	Hit port 8080 on this machine, endpoint /hello

When client and server are both on the same machine, no DNS lookup is needed. localhost always refers to your own computer.

5. Spring Initializr — start.spring.io

Without it, building a Java web app from scratch means doing all of this before writing a single line of business logic:

- Create a proper project folder structure
- Download all required dependency JARs manually
- Configure Spring Boot version
- Configure Java version
- Set up package structure

Spring Initializr generates a ready-made project skeleton (zip file) with all of this done for you. Unzip, open in IDE, start writing business logic immediately.

What is a Dependency?

Any external library your code uses. Example: to connect Java to MySQL, you need the MySQL Connector library. Since your code depends on it → it becomes a **dependency**. You don't write it yourself — you reuse it.

Choices made in Spring Initializr (this lecture):

Option	Choice	Why
Project	Maven	Most popular Java project management tool. Team standard.
Language	Java	Our primary language for this series.
Spring Boot	Latest stable (no SNAPSHOT, no RC)	SNAPSHOT = Work In Progress (buggy). RC = Release Candidate (almost final). Always pick stable.
Packaging	JAR	Modern Spring Boot uses JAR. WAR was the old-style web app format.
Config format	Properties	application.properties file (alternative is YAML — both work fine).
Java Version	21	LTS release — stable, modern, not too new and not too old.
Dependency	Spring Web	Enables REST APIs via Spring MVC. Also embeds Apache Tomcat server automatically.

6. Generated Project Structure

After unzipping and opening in IntelliJ IDEA:

```
src/ main/ java/ com.example.demo/ DemoApplication.java ← Entry point — has the main()
method resources/ static/ ← Static files (images, JS, CSS) templates/ ← HTML templates
application.properties ← App configuration (port, DB settings, etc.) test/ java/
com.example.demo/ DemoApplicationTests.java pom.xml ← Maven config + dependencies
```

DemoApplication.java has a main() method — execution starts here. Don't worry about @SpringBootApplication and SpringApplication.run() yet — covered in later lectures.

7. Writing the Hello World App

Create a new Java class inside com.example.demo package named HelloController:

```
@RestController public class HelloController { @GetMapping("/hello") public String hello()
{ return "Hello World"; } @GetMapping("/bye") public String greetBye() { return "Bye"; } }
```

What each annotation does (preview — full detail in later lectures):

Annotation	What it does
@RestController	Marks this class as a gateway for API endpoints. Spring Boot will scan it for URL mappings.
@GetMapping("/hello")	Maps HTTP GET requests to /hello → calls the hello() method automatically when that URL is hit.
@GetMapping("/bye")	Maps HTTP GET requests to /bye → calls greetBye() method. Note: method name ≠ endpoint name.

The method name and the endpoint name do NOT have to match. @GetMapping("/bye") can call a method named greetBye() — the annotation does the mapping.

8. Running the App & Reading the Logs

Run DemoApplication → watch these key console log lines:

Log Line	What it Means
Tomcat initialized with port 8080	Embedded Tomcat servlet container is starting up on port 8080.
Starting service [Tomcat]	The Tomcat service engine is coming online.
Tomcat started on port(s): 8080	Server is now live and listening for incoming HTTP requests.

Now open browser and visit:

```
localhost:8080/hello → Hello World localhost:8080/bye → Bye
```

Whitelabel Error Page on an unknown endpoint means: 'No mapping found for this URL'. The server IS running — you just haven't created a handler for that path yet.

9. Changing the Default Port

Open src/main/resources/application.properties and add:

```
server.port=9090
```

After restart: Tomcat now listens on port 9090. localhost:8080/hello → 'This site can't be reached'.

localhost:9090/hello → Hello World.

10. Bonus — Returning HTML Tags

You can return HTML directly from the method:

```
return "Hello World";
```

The browser renders the HTML tag — Hello World appears as a large heading. Spring Boot returns whatever the method returns as the raw HTTP response body.

11. Open Questions — Answered in Future Lectures

Question	Covered in...
Who started the Tomcat server?	Spring Boot auto-configuration internals lecture
How did /hello map to the hello() method automatically?	Spring MVC — DispatcherServlet deep dive
What does @RestController actually do internally?	Spring MVC annotations + component scanning lecture
What does SpringApplication.run() do inside?	Spring Core + IoC container lecture

Question	Covered in...
Why learn Spring MVC if Spring Boot handles everything?	Because Spring Boot uses MVC internally — can't debug without knowing the internals

12. Lecture Summary

What we did	How
Created project	Spring Initializr → downloaded zip → opened in IntelliJ IDEA
Added endpoint	Created HelloController.java with @RestController + @GetMapping
Ran the app	DemoApplication.main() → embedded Tomcat auto-starts on port 8080
Tested in browser	localhost:8080/hello → Hello World localhost:8080/bye → Bye
Changed port	server.port=9090 in application.properties
Multiple endpoints	@GetMapping("/bye") on greetBye() → returns 'Bye' string
HTML response	Return "Hello World" from method → browser renders it as heading

Next Lecture: Maven deep-dive — pom.xml, dependencies, build lifecycle